Материалы для самостоятельного изучения

Стандарт PEP8

Лекция 2

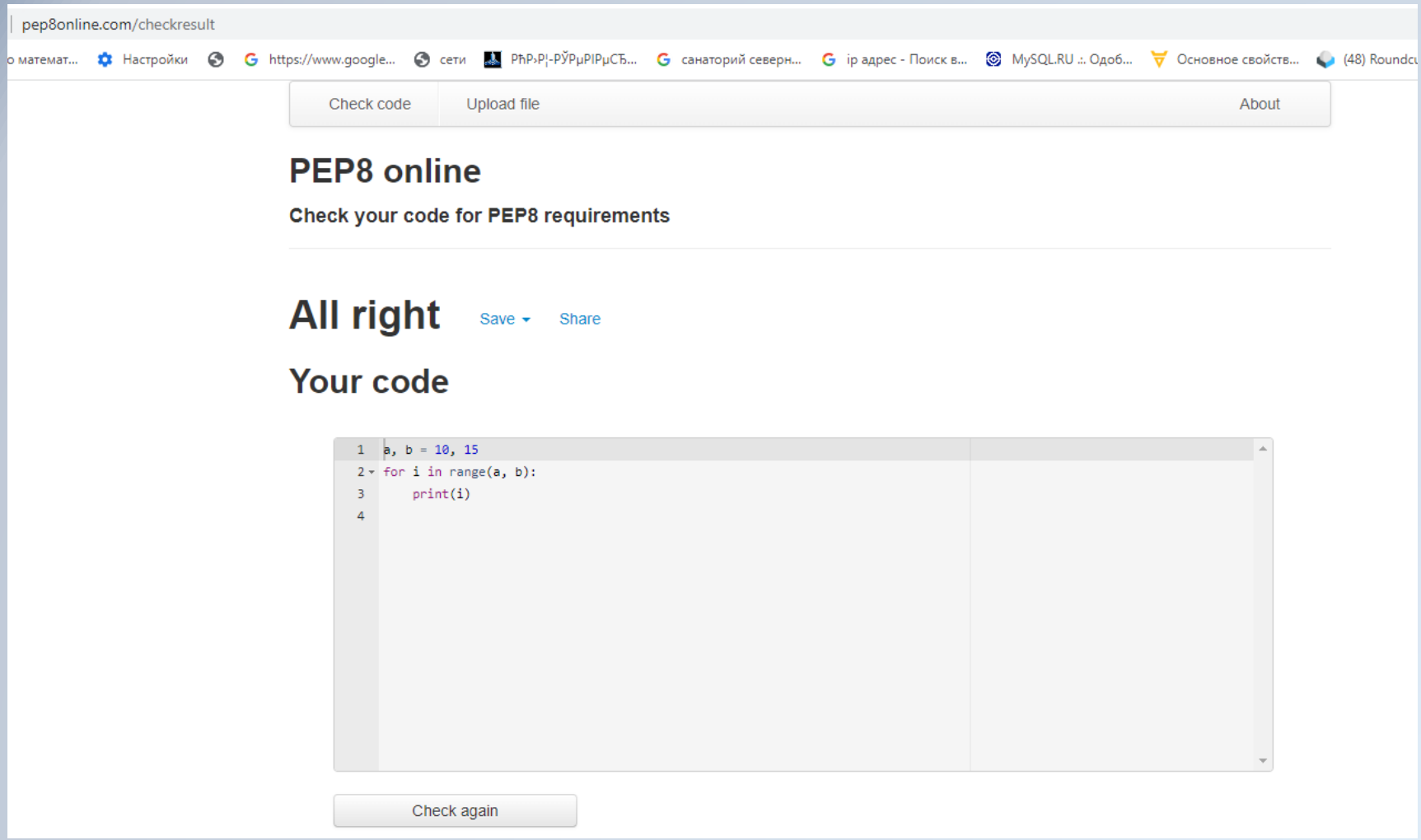
Идеология PEP8

- PEP8 можно определить, как документ, описывающий общепринятый стиль написания кода на языке Python. Python Enhanced Proposal (PEP) - переводится, как заявки по улучшению языка Python.
- Ключевая идея PEP такова: код читается намного больше раз, чем пишется. Собственно, рекомендации о стиле написания кода направлены на то, чтобы улучшить читаемость кода и сделать его согласованным между большим числом проектов. В идеале, весь код будет написан в едином стиле, и любой сможет легко его прочесть (**Гвидо ван Россумом**).
- Это руководство о согласованности и единстве. Согласованность с этим руководством очень важна. Согласованность внутри одного проекта еще важнее. А согласованность внутри модуля или функции – самое важное. Но важно помнить, что иногда это руководство неприменимо, и понимать, когда можно отойти от рекомендаций. Когда вы сомневаетесь, просто посмотрите на другие примеры и решите, какой выглядит лучше (**Гвидо ван Россумом**).

Идеология PEP8

- PEP8 – является ключевым по тому, что создаёт общий для всех питонистов стандарт оформления кода Python, а значит, большинство программ будет написано так, как все привыкли и их будет легче читать, понимать, имправлять.
- В PEP8 описан стандарт оформления кода, направленный на то, чтоб все «говорили на одном языке». Он вобрал в себя опыт ***матёрых*** разработчиков, которые знают, какой стиль оформления наиболее прост для восприятия.
- Соблюдение PEP8 на столько важно, что это первое, на что смотрят работодатели при поиске сотрудников. Если Ваш код не отличается чистотой и читаемостью, то, скорее всего, на его алгоритмическую составляющую даже не будут смотреть.

Проверка кода на соответствие PEP8 online



Внешний вид кода. Отступы

Используйте 4 пробела на каждый уровень отступа.

Продолжительные строки должны выравнивать обернутые элементы либо вертикально, используя неявную линию в скобках (круглых, квадратных или фигурных), либо с использованием висячего отступа. При использовании висячего отступа следует применять следующие соображения: на первой линии не должно быть аргументов, а остальные строки должны четко восприниматься как продолжение линии.

Правильно:

```
# Выровнено по открывающему разделителю
foo = long_function_name(var_one, var_two,
                         var_three, var_four)

# Больше отступов включено для отличения его от остальных
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)
```

Внешний вид кода. Отступы

Неправильно:

```
# Аргументы на первой линии запрещены, если не используется вертикальное
foo = long_function_name(var_one, var_two,
    var_three, var_four)

# Больше отступов требуется, для отличения его от остальных
def long_function_name(
    var_one, var_two, var_three,
    var_four):
    print(var_one)
```

Опционально:

```
# Нет необходимости в большем количестве отступов.
foo = long_function_name(
  var_one, var_two,
  var_three, var_four)
```

Внешний вид кода. Отступы

Закрывающие круглые/квадратные/фигурные скобки в многострочных конструкциях могут находиться под первым не пробельным символом последней строки списка, например:

```
my_list = [
    1, 2, 3,
    4, 5, 6,
    ]
result = some_function_that_takes_arguments(
    'a', 'b', 'c',
    'd', 'e', 'f',
    )
```

либо быть под первым символом строки, начинающей многострочную конструкцию:

```
    4, 5, 6,
]
result = some_function_that_takes_arguments(
    'a', 'b', 'c',
    'd', 'e', 'f',
)
```

Табуляция или пробелы?

Пробелы - самый предпочтительный метод отступов.

Табуляция должна использоваться только для поддержки кода, написанного с отступами с помощью табуляции.

Python 3 запрещает смешивание табуляции и пробелов в отступах.

Python 2 пытается преобразовать табуляцию в пробелы.

Когда вы вызываете интерпретатор Python 2 в командной строке с параметром `-t`, он выдает предупреждения (`warnings`) при использовании смешанного стиля в отступах, а запустив интерпретатор с параметром `-tt`, вы получите в этих местах ошибки `(errors)`. Эти параметры очень рекомендуются!

Максимальная длина строки

Ограничьте длину строки максимум 79 символами.

Предпочтительный способ переноса длинных строк является использование подразумеваемых продолжений строк Python внутри круглых, квадратных и фигурных скобок. Длинные строки могут быть разбиты на несколько строк, обернутые в скобки. Это предпочтительнее использования обратной косой черты для продолжения строки.

Обратная косая черта все еще может быть использована время от времени. Например, длинная конструкция *with* не может использовать неявные продолжения, так что обратная косая черта является приемлемой:

```
with open('/path/to/some/file/you/want/to/read') as file_1, \
        open('/path/to/some/file/being/written', 'w') as file_2:
    file_2.write(file_1.read())
```

Максимальная длина строки

- Ещё один случай - assert.
- Сделайте правильные отступы для перенесённой строки. Предпочтительнее вставить перенос строки после логического оператора, но не перед ним. Например:

```
class Rectangle(Blob):

    def __init__(self, width, height,
                 color='black', emphasis=None, highlight=0):
        if (width == 0 and height == 0 and
                color == 'red' and emphasis == 'strong' or
                highlight > 100):
            raise ValueError("sorry, you lose")
        if width == 0 and height == 0 and (color == 'red' or
                                           emphasis is None):
            raise ValueError("I don't think so -- values are %s, %s" %
                             (width, height))
        Blob.__init__(self, width, height,
                      color, emphasis, highlight)
```

Отделяйте функции верхнего уровня и определения классов двумя пустыми строками.

Определения методов внутри класса разделяются одной пустой строкой.

Дополнительные пустые строки возможно использовать для разделения различных групп похожих функций. Пустые строки могут быть опущены между несколькими связанными однострочниками (например, набор фиктивных реализаций).

Используйте пустые строки в функциях, чтобы указать логические разделы.

Python расценивает символ *control+L* как незначащий (*whitespace*), и вы можете использовать его, потому что многие редакторы обрабатывают его как разрыв страницы — таким образом логические части в файле будут на разных страницах. Однако, не все редакторы распознают *control+L* и могут на его месте отображать другой символ.

Кодировка исходного файла

- Кодировка Python должна быть UTF-8 (ASCII в Python 2).
- Файлы в ASCII (Python 2) или UTF-8 (Python 3) не должны иметь объявления кодировки.
- В стандартной библиотеке, нестандартные кодировки должны использоваться только для целей тестирования, либо когда комментарий или строка документации требует упомянуть имя автора, содержащего не ASCII символы; в остальных случаях использование \x, \u, \U или \N - наиболее предпочтительный способ включить не ASCII символы в строковых литералах.
- Начиная с версии python 3.0 в стандартной библиотеке действует следующее соглашение: все идентификаторы обязаны содержать только ASCII символы, и означать английские слова везде, где это возможно (во многих случаях используются сокращения или неанглийские технические термины). Кроме того, строки и комментарии тоже должны содержать лишь ASCII символы. Исключения составляют: (a) test case, тестирующий не-ASCII особенности программы, и (б) имена авторов. Авторы, чьи имена основаны не на латинском алфавите, должны транслитерировать свои имена в латиницу.

Импорты

- Каждый импорт, как правило, должен быть на отдельной строке.

  Правильно:

```
import os
import sys
```

  Неправильно:

```
import sys, os
```

  В то же время, можно писать так:

```
from subprocess import Popen, PIPE
```

Кодировка исходного файла

- Кодировка Python должна быть UTF-8 (ASCII в Python 2).
- Файлы в ASCII (Python 2) или UTF-8 (Python 3) не должны иметь объявления кодировки.
- В стандартной библиотеке, нестандартные кодировки должны использоваться только для целей тестирования, либо когда комментарий или строка документации требует упомянуть имя автора, содержащего не ASCII символы; в остальных случаях использование \x, \u, \U или \N - наиболее предпочтительный способ включить не ASCII символы в строковых литералах.
- Начиная с версии python 3.0 в стандартной библиотеке действует следующее соглашение: все идентификаторы обязаны содержать только ASCII символы, и означать английские слова везде, где это возможно (во многих случаях используются сокращения или неанглийские технические термины). Кроме того, строки и комментарии тоже должны содержать лишь ASCII символы. Исключения составляют: (a) test case, тестирующий не-ASCII особенности программы, и (б) имена авторов. Авторы, чьи имена основаны не на латинском алфавите, должны транслитерировать свои имена в латиницу.

Импорт

- Импорты всегда помещаются в начале файла, сразу после комментариев к модулю и строк документации, и перед объявлением констант.
- Импорты должны быть сгруппированы в следующем порядке:
  - импорты из стандартной библиотеки
  - импорты сторонних библиотек
  - импорты модулей текущего проекта
- Вставляйте пустую строку между каждой группой импортов.
- Указывайте спецификации __all__ после импортов.
- Рекомендуется абсолютное импортирование, так как оно обычно более читаемо и ведет себя лучше (или, по крайней мере, даёт понятные сообщения об ошибках) если импортируемая система настроена неправильно (например, когда каталог внутри пакета заканчивается на sys.path):

Импорт

- Когда вы импортируете класс из модуля, вполне можно писать вот так:

```
from myclass import MyClass
from foo.bar.yourclass import YourClass
```

Если такое написание вызывает конфликт имен, тогда пишите:

```
import myclass
import foo.bar.yourclass
```

И используйте "myclass.MyClass" и "foo.bar.yourclass.YourClass".

Пробелы в выражениях и инструкциях

Избегайте использования пробелов в следующих ситуациях:

- Непосредственно внутри круглых, квадратных или фигурных скобок.

Правильно:

```
spam(ham[1], {eggs: 2})
```

Неправильно

```
spam( ham[ 1 ], { eggs: 2 } )
```

- Непосредственно перед запятой, точкой с запятой или двоеточием:

Правильно:

```
if x == 4: print(x, y); x, y = y, x
```

Неправильно

```
if x == 4 : print(x , y) ; x , y = y , x
```

Пробелы в выражениях и инструкциях

- Сразу перед открывающей скобкой, после которой начинается список аргументов при вызове функции:

  Правильно:

```
spam(1)
```

  Неправильно:

```
spam (1)
```

- Сразу перед открывающей скобкой, после которой следует индекс или срез:

  Правильно:

```
dict['key'] = list[index]
```

  Неправильно:

```
dict ['key'] = list [index]
```

Пробелы в выражениях и инструкциях

- Использование более одного пробела вокруг оператора присваивания (или любого другого) для того, чтобы выровнять его с другим:

  Правильно:

```
x = 1
y = 2
long_variable = 3
```

  Неправильно:

```
x             = 1
y             = 2
long_variable = 3
```

Другие рекомендации

- Всегда окружайте эти бинарные операторы одним пробелом с каждой стороны: присваивания *(=, +=, -=* и другие), сравнения *(==, <, >, !=, <>, <=, >=, in, not in, is, is not)*, логические (*and, or, not*).
- Если используются операторы с разными приоритетами, попробуйте добавить пробелы вокруг операторов с самым низким приоритетом. Используйте свои собственные суждения, однако, никогда не используйте более одного пробела, и всегда используйте одинаковое количество пробелов по обе стороны бинарного оператора.

Правильно:

```
i = i + 1
submitted += 1
x = x*2 - 1
hypot2 = x*x + y*y
c = (a+b) * (a-b)
```

Неправильно

```
i=i+1
submitted +=1
x = x * 2 - 1
hypot2 = x * x + y * y
c = (a + b) * (a - b)
```

Другие рекомендации

- Не используйте пробелы вокруг знака =, если он используется для обозначения именованного аргумента или значения параметров по умолчанию.

  Правильно:

```
def complex(real, imag=0.0):
    return magic(r=real, i=imag)
```

  Неправильно:

```
def complex(real, imag = 0.0):
    return magic(r = real, i = imag)
```

Другие рекомендации

- Не используйте составные инструкции (несколько команд в одной строке).

  Правильно:

```
if foo == 'blah':
    do_blah_thing()
do_one()
do_two()
do_three()
```

  Неправильно:

```
if foo == 'blah': do_blah_thing()
do_one(); do_two(); do_three()
```

Другие рекомендации

- Иногда можно писать тело циклов while, for или ветку if в той же строке, если команда короткая, но если команд несколько, никогда так не пишите. А также избегайте длинных строк!

  Точно неправильно:

```
if foo == 'blah': do_blah_thing()
for x in lst: total += x
while t < 10: t = delay()
```

  Вероятно, неправильно:

```
if foo == 'blah': do_blah_thing()
else: do_non_blah_thing()

try: something()
finally: cleanup()

do_one(); do_two(); do_three(long, argument,
                             list, like, this)

if foo == 'blah': one(); two(); three()
```

Комментарии

- Комментарии, противоречащие коду, хуже, чем отсутствие комментариев. Всегда исправляйте комментарии, если меняете код!
- Комментарии должны являться законченными предложениями. Если комментарий — фраза или предложение, первое слово должно быть написано с большой буквы, если только это не имя переменной, которая начинается с маленькой буквы (никогда не изменяйте регистр переменной!).
- Если комментарий короткий, можно опустить точку в конце предложения. Блок комментариев обычно состоит из одного или более абзацев, составленных из полноценных предложений, поэтому каждое предложение должно оканчиваться точкой.
- Ставьте два пробела после точки в конце предложения.
- Программисты, которые не говорят на английском языке, пожалуйста, пишите комментарии на английском, если только вы не уверены на 120%, что ваш код никогда не будут читать люди, не знающие вашего родного языка.

Блоки комментариев

- Блок комментариев обычно объясняет код (весь, или только некоторую часть), идущий после блока, и должен иметь тот же отступ, что и сам код. Каждая строчка такого блока должна начинаться с символа # и одного пробела после него (если только сам текст комментария не имеет отступа).
- Абзацы внутри блока комментариев разделяются строкой, состоящей из одного символа #.

"Встрочные" комментарии

- Старайтесь реже использовать подобные комментарии.
- Такой комментарий находится в той же строке, что и инструкция. "Встрочные" комментарии должны отделяться по крайней мере двумя пробелами от инструкции. Они должны начинаться с символа # и одного пробела.
- Комментарии в строке с кодом не нужны и только отвлекают от чтения, если они объясняют очевидное. Не пишите вот так:

```
x = x + 1 # Increment x
```

Впрочем, такие комментарии иногда полезны:

```
x = x + 1 # Компенсация границы
```

Описание: Стили имен

Существует много разных стилей. Поможем вам распознать, какой стиль именования используется, независимо от того, для чего он используется.

Обычно различают следующие стили:

- b (одиночная маленькая буква)
- B (одиночная заглавная буква)
- lowercase (слово в нижнем регистре)
- lower_case_with_underscores (слова из маленьких букв с подчеркиваниями)
- UPPERCASE (заглавные буквы)
- UPPERCASE_WITH_UNDERSCORES (слова из заглавных букв с подчеркиваниями)
- CapitalizedWords (слова с заглавными буквами, или CapWords, или CamelCase). Замечание: когда вы используете аббревиатуры в таком стиле, пишите все буквы аббревиатуры заглавными — HTTPServerError лучше, чем HttpServerError.
- mixedCase (отличается от CapitalizedWords тем, что первое слово начинается с маленькой буквы)
- Capitalized_Words_With_Underscores (слова с заглавными буквами и подчеркиваниями — уродливо!)

Предписания: соглашения по именованию

Имена, которых следует избегать

- Никогда не используйте символы l (маленькая латинская буква «эль»), O (заглавная латинская буква «о») или I (заглавная латинская буква «ай») как однобуквенные идентификаторы.
- В некоторых шрифтах эти символы неотличимы от цифры один и нуля. Если очень нужно l, пишите вместо неё заглавную L.

Предписания: соглашения по именованию

Имена модулей и пакетов

- Модули должны иметь короткие имена, состоящие из маленьких букв. Можно использовать символы подчеркивания, если это улучшает читабельность. То же самое относится и к именам пакетов, однако в именах пакетов не рекомендуется использовать символ подчёркивания.
- Так как имена модулей отображаются в имена файлов, а некоторые файловые системы являются нечувствительными к регистру символов и обрезают длинные имена, очень важно использовать достаточно короткие имена модулей — это не проблема в Unix, но, возможно, код окажется непереносимым в старые версии Windows, Mac, или DOS.
- Когда модуль расширения, написанный на C или C++, имеет сопутствующий python-модуль (содержащий интерфейс высокого уровня), C/C++ модуль начинается с символа подчеркивания, например, _socket.

Предписания: соглашения по именованию

Имена классов

- Имена классов должны обычно следовать соглашению CapWords.
- Вместо этого могут использоваться соглашения для именования функций, если интерфейс документирован и используется в основном как функции.
- Обратите внимание, что существуют отдельные соглашения о встроенных именах: большинство встроенных имен - одно слово (либо два слитно написанных слова), а соглашение CapWords используется только для именования исключений и встроенных констант.

Имена исключений

- Так как исключения являются классами, к исключениями применяется стиль именования классов. Однако вы можете добавить Error в конце имени (если, конечно, исключение действительно является ошибкой).

Предписания: соглашения по именованию

Имена глобальных переменных

- Будем надеяться, что глобальные переменные используются только внутри одного модуля. Руководствуйтесь теми же соглашениями, что и для имен функций.
- Добавляйте в модули, которые написаны так, чтобы их использовали с помощью from M import *, механизм __all__, чтобы предотвратить экспортирование глобальных переменных. Или же, используйте старое соглашение, добавляя перед именами таких глобальных переменных один символ подчеркивания (которым вы можете обозначить те глобальные переменные, которые используются только внутри модуля).

Имена функций

- Имена функций должны состоять из маленьких букв, а слова разделяться символами подчеркивания — это необходимо, чтобы увеличить читабельность.
- Стиль mixedCase допускается в тех местах, где уже преобладает такой стиль, для сохранения обратной совместимости.

Предписания: соглашения по именованию

Константы

- Константы обычно объявляются на уровне модуля и записываются только заглавными буквами, а слова разделяются символами подчеркивания. Например: MAX_OVERFLOW, TOTAL.

Ссылка для дальнейшего изучения стандарта:

https://peps.python.org/pep-0008/